

A large red square with a white border, centered on a white background. Inside the square, the text "Ntier - Clean Architecture" is written in white.

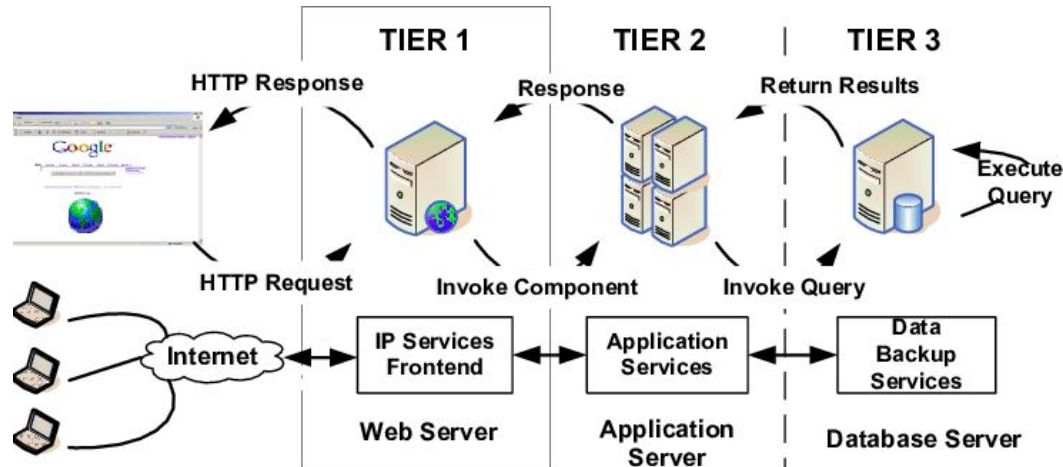
Ntier - Clean Architecture

Segmentation d'un projet multitier

- Il est important de découper un projet informatique en blocs/couches/layers bien distincts
- En informatique, le principe de séparation des responsabilités à cours à tous les niveaux : au sein des fonctions, au sein des classes et aussi des grands ensembles d'une application que l'on appelle Layer
- Chaque Layer de plus haut niveau possède, son serveur ou son instance propre, l'objectif étant de séparer complètement ces layers, et de les modifier ou les faire s'échelonner de manière complètement autonomes

Architecture n-tiers

- 3 Layers, sur 3 serveurs/terminaux/docker différents
 - La couche Display/Présentation
 - La couche Business
 - La couche Base de données



Intérêts

- Le segmentation permet une plus grande maintenabilité de la structure
- Chaque couche peut être modifiée indépendamment sans crainte pour les autres
- L'avantage majeure est la scalabilité du projet
 - L'architecture de chaque couche peut être modifiée pour s'adapter à un nouveau besoin
 - La puissance de chaque peut être modifiée indépendamment pour répondre à, par exemple, une augmentation de charge
 - La maintenance et la pérennité sont également facilités l'architecture ntier elle permet la mise en oeuvre de système plus complexe

Organiser sa couche Business

- Dans le cas d'un projet Php Symfony ou Laravel, la couche business se situera sur le serveur du framework
- L'objectif est clair : il faut, dans cette couche, différencier le code **INFRASTRUCTURE** liés aux décisions techniques, framework, et librairies tierces, le code de **PRÉSENTATION**, liés au vues, et le code **MÉTIER**, qui va contenir le coeur de votre application, la valeur ajoutée de celle-ci
- Votre code métier doit être complète **IMPERMÉABLE** à toute les autres couche en utilisant les principes de la programmation **SOLID**
- Il faut, in fine, pouvoir changer de Framework, ou d'ORM, sans avoir à changer un caractère de sa couche métier

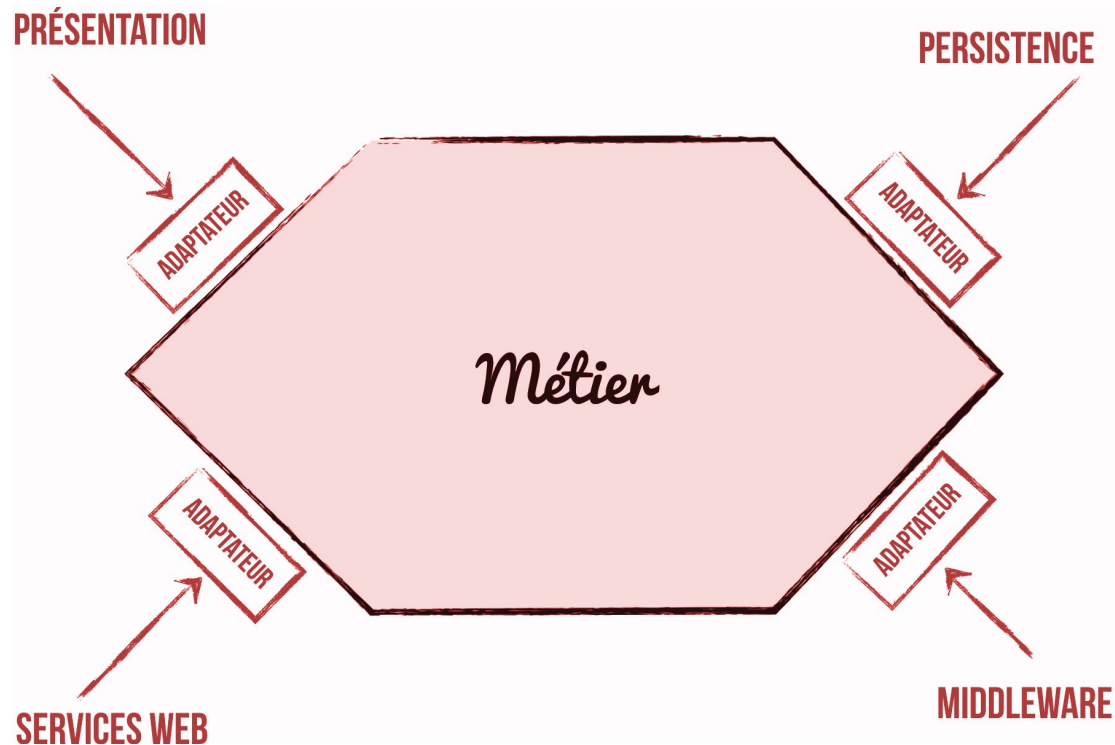
Clean Architecture – Historique & Intérêts

- Détaillée en 2012 par Robert C. Martin, a.k.a. Uncle Bob, également créateur des principes de la programmation SOLID
- Design pattern créer afin de rendre sa couche business
 - **Isolée, Autonome, Moins couplée à son environnement**
 - Beaucoup plus testable
 - Beaucoup plus facilement maintenable

<https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>

- Autres concepts similaires ou dérivant
 - Architecture hexagonale
 - Architecture en Onion
 - Domain Driven Architecture

Architecture Hexagonale du code Métier



Notion PORT - ADAPTATEUR

- L'architecture hexagonale fonctionne selon le principe de port adaptateur
- Il s'agit en réalité, d'une utilisation systématique du principe **SOLID** d'inversion de dépendance lorsque les classes **MÉTIER**s doivent utiliser des classes de l'**INFRASTRUCTURE** (comme des repositories, ou des clients http tels que guzzle ...)
- Le principe est très simple
 - **PORT** = INTERFACE MÉTIER, se trouvant au sein du code métier
 - **ADAPTATEUR** = CLASSE INFRASTRUCTURE IMPLÉMENTANT L'INTERFACE se trouvant au sein du code infra

EXEMPLE DE CODE

Notre classe Métier Quizz a besoin de communiquer à la base de données. Plutôt que d'utiliser directement les repositories créés dans l'infrastructure, je créé des interfaces, se trouvant dans mon code métier que mes repositories infra devront OBLIGATOIREMENT implémenter. Ici, RepositoryInterface, QuestionRepositoryInterface et ParserInterface sont des ports

```
<?php namespace App\Quizz\Quizz;

use \App\Quizz\Question\RepositoryInterface as QuestionRepositoryInterface;
class Quizz
{
    protected array $questions = [];
    protected RepositoryInterface $repository;
    protected QuestionRepositoryInterface $questionRepository;
    protected ParserInterface $parser;

    public function __construct(
        RepositoryInterface $repository,
        ParserInterface $parser,
        QuestionRepositoryInterface $questionRepository
    )
    {
        $this->parser = $parser;
        $this->repository = $repository;
        $this->questionRepository = $questionRepository;
        $this->questions = $this->parser->parse($this->repository->fetch(), $questionRepository);
    }

    public function save(){
        foreach ($this->questions as $question) {
            $question->save();
        }
    }
}
```

EXEMPLE DE CODE

```
<?php namespace App\Quizz\Quizz;  
  
interface RepositoryInterface{  
    public function fetch():array;  
}
```

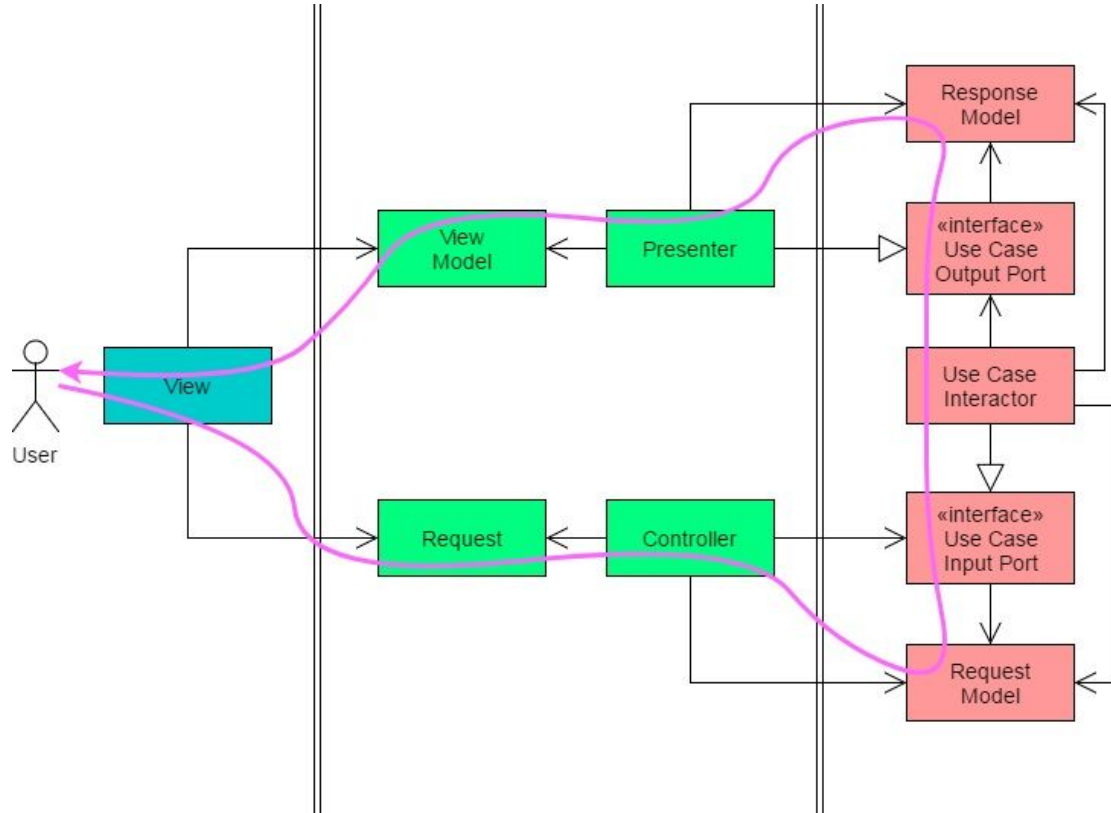
```
public function handle()  
{  
    $quizz = new Quizz\Quizz(new QuizzRepository(), new Quizz\Parser(), new QuestionRepository());  
    /* @var $quizz Quizz\Quizz*/  
    $quizz->save();  
}
```

```
<?php namespace App\Repositories;  
  
use App\Presenters\QuizzPresenter;  
use App\Quizz\Quizz\RepositoryInterface;  
  
class QuizzRepository implements RepositoryInterface   
{  
    protected string $file;  
    protected $presenter;  
  
    public function __construct($file = 'questions.json')  
    {  
        if(!file_exists($file))  
            throw new \Exception('file does not exists');  
        $this->file = $file;  
        $this->presenter = new QuizzPresenter();  
    }  
  
    public function fetch():array{  
        $content = file_get_contents($this->file, true);  
        return $this->presenter->present(json_decode($content));  
    }  
}
```

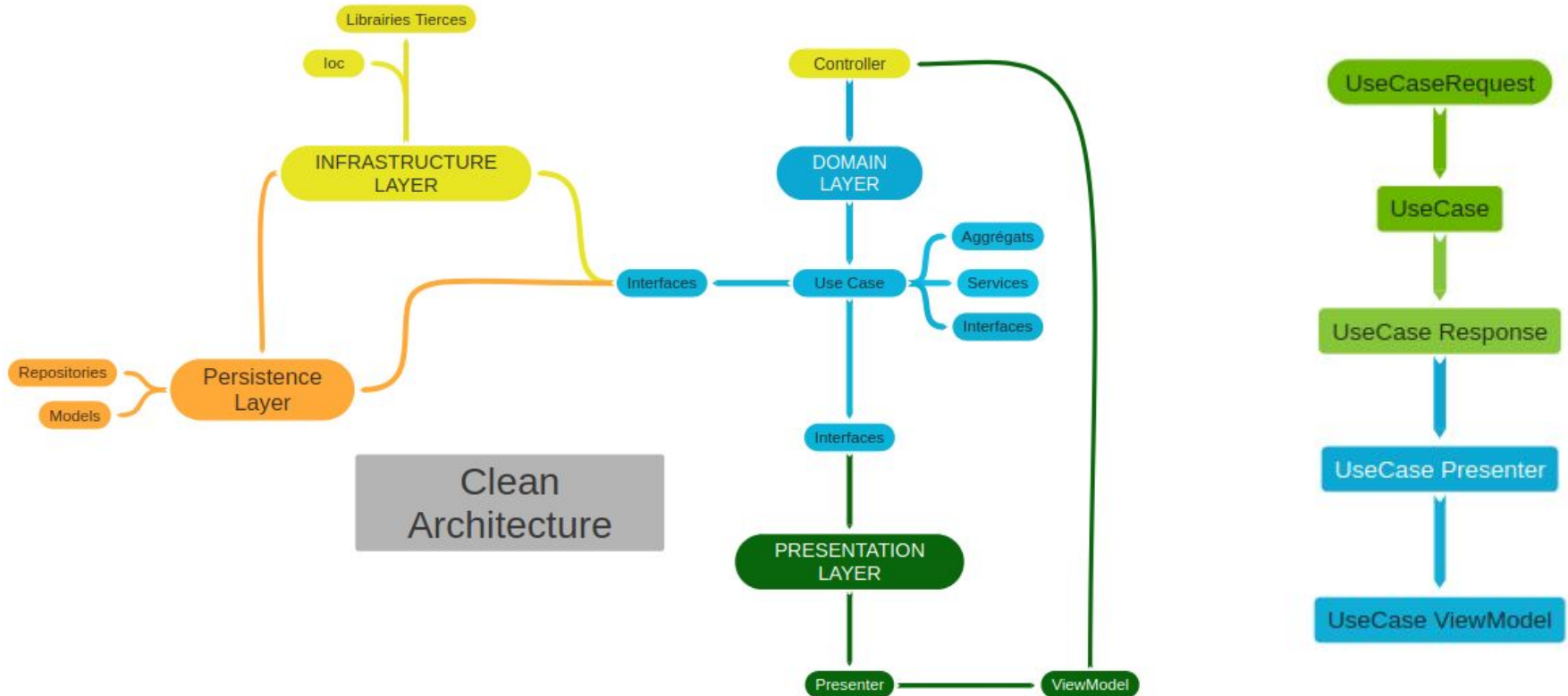
Use Case, Presenter et ViewModel

- Un projet Clean Architecture se pense en terme de UseCase
- Concrètement, un UseCase est une Classe métier représentant une action de votre projet. Il s'agira de votre point d'entrée vers votre code métier, la classe **UseCase** prendra en paramètre une classe **Request** (simple Data Object) contenant les informations nécessaires au UseCase Et retournera un objet **Response**. Cet objet sera envoyé au **Presenter** qui créera un **ViewModel** Qui sera envoyé à la vue
- Un use case est un scénario, par exemple : DisplayCartUseCase, RegisterCarUseCase ...

Use Case, Presenter et VueModel



Use Case, Presenter et VueModel



Segmentation classique d'un projet PHP CA

